

Lab program-13:

13. Construct a C program to implement various memory allocation strategies.

```
C lab_13.c > main()
1  #include <stdio.h>
2
3  // Descending Sort
4  void rsort(int a[], int n)
5  {
6      int i, j, t;
7
8      for (i = 0; i < n - 1; i++)
9      {
10         for (j = i + 1; j < n; j++)
11         {
12             if (a[i] < a[j])
13             {
14                 t = a[i];
15                 a[i] = a[j];
16                 a[j] = t;
17             }
18         }
19     }
20 }
21
22 // Ascending Sort
23 void sort(int a[], int n)
24 {
25     int i, j, t;
26
27     for (i = 0; i < n - 1; i++)
28     {
29         for (j = i + 1; j < n; j++)
30         {
31             if (a[i] > a[j])
32             {
33                 t = a[i];
34                 a[i] = a[j];
35                 a[j] = t;
36             }
37         }
38     }
39 }
40
41 // Best Fit Allocation
42 void bestfit(int mp[], int p[], int m, int n)
43 {
44     int j = 0;
45
46     for (int i = 0; i < n && j < m; i++)
47     {
48         if (mp[i] >= p[j])
49         {
50             printf("\nProcess %d fits in Memory Partition %d",
51                 p[j], mp[i]);
52             mp[i] -= p[j];
53             j++;
54             i--;
55         }
56     }
57
58     for (int i = j; i < m; i++)
59     {
60         printf("\nProcess %d must wait.", p[i]);
61     }
62 }
63
64 // First Fit
65 void firstfit(int mp[], int p[], int m, int n)
66 {
67     bestfit(mp, p, m, n);
68 }
69
70
```

```

71 // Worst Fit
72 void worstfit(int mp[], int p[], int m, int n)
73 {
74     rsort(mp, n);
75     sort(p, m);
76     bestfit(mp, p, m, n);
77 }
78
79 int main()
80 {
81     int m, n;
82     int mp[20], p[20];
83     int ch;
84
85     printf("Number of Memory Partitions: ");
86     scanf("%d", &n);
87
88     printf("Number of Processes: ");
89     scanf("%d", &m);
90
91     printf("\nEnter the Memory Partition Sizes:\n");
92
93     for (int i = 0; i < n; i++)
94     {
95         scanf("%d", &mp[i]);
96     }
97
98     printf("\nEnter the Process Sizes:\n");
99
100    for (int i = 0; i < m; i++)
101    {
102        scanf("%d", &p[i]);
103    }
104
105    printf("\n1. First Fit");
106    printf("\n2. Best Fit");
107    printf("\n3. Worst Fit");
108    printf("\nEnter your Choice: ");
109    scanf("%d", &ch);
110
111    switch (ch)
112    {
113    case 1:
114        firstfit(mp, p, m, n);
115        break;
116
117    case 2:
118        sort(mp, n);
119        sort(p, m);
120        bestfit(mp, p, m, n);
121        break;
122
123    case 3:
124        worstfit(mp, p, m, n);
125        break;
126
127    default:
128        printf("Invalid Choice!");
129    }
130
131    return 0;
132 }

```

OUTPUT:

```
PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_13.exe
Number of Memory Partitions: 4
Number of Processes: 4

Enter the Memory Partition Sizes:
2
3
4
2

Enter the Process Sizes:
4
3
2
1

1. First Fit
2. Best Fit
3. Worst Fit
Enter your Choice: 2

Process 1 fits in Memory Partition 2
Process 2 fits in Memory Partition 2
Process 3 fits in Memory Partition 3
Process 4 fits in Memory Partition 4
```